Trabalho Final

Aplicação Web com Análise de Dados e Machine Learning

Tópicos Especiais em Ciência da Computação

5,0 pontos • Grupos de 3 a 4 alunos

O desafio do projeto

Problema real	Escolher uma pergunta que possa ser respondida com dados tabulares e aprendizado supervisionado.
Produto completo	Construir uma aplicação Flask com upload, análise, gráficos e previsão no navegador.
Profundidade esperada	Justificar decisões, interpretar resultados e demonstrar domínio individual do código.

Escopo obrigatório e limites

Integração obrigatória

Flask e HTML
Upload de CSV e análise com Pandas
Pelo menos 3 visualizações relevantes
Teste de no mínimo 2 modelos do scikit-learn
Avaliação e previsão por formulário

Recursos que não são exigidos

Selenium ou Playwright
Mapas ou gráficos interativos avançados
Escolha de vários modelos pelo usuário
Retreinamento dinâmico obrigatório
Configuração de hiperparâmetros
Deep learning

Fluxo visual do sistema

ANÁLISE NO NAVEGADOR

Upload CSV → **Validação + Pandas** → **Análise + 3 gráficos** → **Página de resultados**

MODELO E PREVISÃO

X / y + treino e teste → **Comparar + avaliar** → **Formulário + predict** → **Previsão em HTML**

O Flask conecta a interface aos dados, às análises e ao modelo treinado.

Upload de CSV e validação

Formato esperado	O grupo deve definir e explicar as colunas aceitas pela aplicação. O usuário poderá enviar arquivos com registros diferentes, desde que respeitem esse formato.
Validação do arquivo	Verificar arquivo vazio, formato, separador, colunas obrigatórias e valores incompatíveis. Mostrar mensagens compreensíveis.
Resultado visível	Exibir uma amostra, quantidade de registros e colunas. Atualizar as análises e os gráficos a partir do arquivo enviado.

Leitura, limpeza e análise com Pandas

Diagnóstico	Verificar tipos, valores ausentes, duplicatas e valores impossíveis. Apresentar o tamanho da base antes e depois.
Tratamento justificado	Explicar exclusões, conversões e preenchimentos. Preservar o CSV original e registrar as transformações.
Análise do problema	Usar estatísticas e agrupamentos para responder perguntas. Explicar ao menos três achados sobre o conjunto de dados.

Pelo menos 3 visualizações relevantes

Distribuição	Exemplo: histograma do preço ou da idade. Explicar concentração, dispersão e valores extremos.
Comparação	Exemplo: barras por categoria ou por classe do alvo. Explicar diferenças e possíveis desequilíbrios.
Relação	Exemplo: dispersão entre área e preço ou boxplot por classe. Relacionar a análise à pergunta do projeto.

Cada gráfico precisa de título, eixos legíveis e interpretação. A matriz de confusão é adicional.

Definição de X e y

X: variáveis de entrada

Características disponíveis no momento da previsão.

Exemplo: área, quartos e idade do imóvel.

O grupo explica por que cada variável pode ajudar a responder à pergunta.

y: o alvo

Resposta que o modelo aprenderá a prever.

Exemplo: preço de venda.

O alvo não entra em X. IDs, respostas futuras e variáveis que revelam y devem ser excluídos.

Modelos de Machine Learning permitidos

Classificação

Decision Tree
KNN
Logistic Regression
Random Forest
Naive Bayes

Regressão

Linear Regression
Decision Tree Regressor
Random Forest Regressor

O grupo NÃO precisa usar todos. Deve pesquisar modelos adequados ao seu tema.

Comparação obrigatória de modelos

No mínimo dois	Pesquisar e testar pelo menos dois modelos permitidos, compatíveis com o mesmo problema e o tipo de alvo.
Resultados comparáveis	Usar a mesma divisão de dados e métricas adequadas. Apresentar os resultados de cada modelo e interpretar os erros.
Escolha final	Justificar o modelo escolhido para a aplicação final, considerando os resultados, os tipos de erro e as limitações.

Não é obrigatório criar uma interface para configurar hiperparâmetros.

Treinamento, teste e prevenção de vazamento

Separação

Usar train_test_split e registrar a proporção e o random_state. Exemplo: 80% para treino e 20% para teste.

Aprendizado

Treinar com X_train e y_train. Ajustar codificações e preenchimentos aprendidos somente no conjunto de treino.

Avaliação honesta

Gerar previsões para X_test e comparar com y_test. Não escolher variáveis ou regras para decorar o teste.

Overfitting: desempenho muito bom no treino e pior em dados que o modelo não viu.

Exemplo mínimo de classificação

```
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier
from sklearn.metrics import accuracy_score, confusion_matrix

X = df[["idade", "renda"]]  # entradas já preparadas
y = df["comprou"]           # resposta conhecida
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.20, random_state=42, stratify=y
)
modelo = DecisionTreeClassifier(max_depth=4, random_state=42)
modelo.fit(X_train, y_train)
y_pred = modelo.predict(X_test)
acuracia = accuracy_score(y_test, y_pred)
matriz = confusion_matrix(y_test, y_pred)
```

Exemplo de um modelo. No trabalho, é obrigatório testar e comparar pelo menos dois.

Avaliação em classificação

Acurácia no teste

Usar accuracy_score(y_test, y_pred).

Acurácia = acertos ÷ total de exemplos.

Informar o tamanho do teste e a distribuição das classes. Comparar com a proporção da classe mais frequente.

Matriz de confusão

Usar confusion_matrix(y_test, y_pred).

Mostrar nomes das classes e identificar os eixos real e previsto.

Explicar quais erros aparecem e por que eles importam no problema.

Podem usar estas métricas e pesquisar outras, se necessário ao problema.

Avaliação em regressão

Pesquisa obrigatória	Pesquisar uma métrica apropriada ao problema, justificar a escolha e usá-la para comparar os modelos.
Exemplo de métrica	MAE é uma possibilidade: mede o erro absoluto médio na unidade do alvo. Não é a única métrica permitida.
Interpretação	Explicar o significado dos valores obtidos e dos erros no contexto do tema. Acurácia e matriz de confusão não se aplicam.

Formulário e previsão dentro da aplicação

Entrada em HTML	Criar campos correspondentes às variáveis X, com rótulos, unidades e opções coerentes para as categorias.
Processamento no Flask	Validar valores, construir uma linha com as mesmas colunas e aplicar o mesmo preparo usado no treinamento.
Saída no navegador	Executar predict() e apresentar a categoria ou o valor previsto. Exibir mensagem amigável para entradas inválidas.

Coerência entre dados, modelo e interface

Modelo disponível	Treinar em um script separado e carregar o modelo no Flask é permitido. Treinar uma vez na aplicação também é possível.
Upload e modelo	O CSV enviado atualiza análises e gráficos. Se o modelo permanecer fixo, informar qual base e versão foram usadas no treino.
Mesma preparação	Preservar nomes, ordem, tipos e codificação das variáveis. Guardar o pré-processamento junto ao modelo, quando necessário.

Estrutura sugerida do projeto

```
projeto/
  app.py
  dados.py
  modelo.py
  treinar.py
  templates/
    index.html
    analise.html
    previsao.html
  static/
  data/
  models/
  requirements.txt
  README.md
```

Responsabilidades

app.py: rotas e integração
dados.py: leitura, limpeza e análise
modelo.py: preparação e previsão
treinar.py: treino e avaliação

templates/: páginas HTML
data/: CSV de exemplo
models/: modelo, se houver persistência
README: instalação e uso

Etapas de desenvolvimento

1. Proposta	Definir equipe, tema, dataset, pergunta, X e y. Obter aprovação antes de desenvolver.
2. Dados e modelo	Explorar e limpar os dados, criar gráficos, separar treino/teste, comparar pelo menos dois modelos e justificar a escolha final.
3. Integração	Construir páginas Flask, upload, análise e formulário. Conectar a previsão ao modelo.
4. Revisão	Testar o fluxo completo, documentar, revisar contribuições e ensaiar a defesa individual.

Entregáveis

Repositório completo	Código organizado e comentado, HTML e recursos utilizados. Incluir requirements.txt e arquivos necessários à execução.
Dados e evidências	CSV de exemplo ou instrução reproduzível de obtenção, fonte, dicionário de dados e resultados da avaliação.
README e apresentação	Explicar instalação, execução, esquema do CSV, X/y, modelos testados, comparação, escolha final, métricas, limitações e participação de cada integrante.

Entrega: link do GitHub no Blackboard. Prazo e data de apresentação: a definir pelo professor.

Critérios de avaliação: total de 5,0 pontos

Critério	Pontos
Aplicação Flask e funcionamento	0,75
Tratamento e análise dos dados	0,75
Visualizações	0,50
Machine Learning	1,25
Previsão integrada	0,50
Organização e documentação	0,25
Apresentação e defesa individual	1,00
TOTAL	**5,00**

Evidências para a pontuação técnica

Flask + dados	0,75: rotas, upload e mensagens funcionam. 0,75: limpeza justificada e análises corretas, com achados interpretados.
Gráficos + previsão	0,50: três gráficos relevantes e legíveis. 0,50: formulário validado gera previsão real no navegador.
ML + documentação	1,25: X/y, treino/teste, comparação de 2 ou mais modelos, avaliação e escolha justificada. 0,25: código organizado e README reproduzível.

Apresentação e defesa individual: 1,00 ponto

Demonstração: 0,25	Apresentar problema, executar upload, explicar gráficos, mostrar avaliação e realizar uma nova previsão.
Defesa individual: 0,75	Cada integrante explica, em linguagem própria, o código, as variáveis, como os modelos funcionam e por que foram escolhidos.
Participação de todos	Todos explicam treino/teste, como avaliaram os modelos e interpretam resultados e erros. A nota individual pode variar.

Sem explicar o código na defesa: nota individual do trabalho = 0,0 de 5,0.

Regras de execução e autoria

Equipe	Equipes com no mínimo 3 e no máximo 4 alunos. Registrar integrantes e responsabilidades no README.
Autoria e fontes	Citar fontes, códigos adaptados e ferramentas, inclusive IA. Cada aluno deve demonstrar participação e compreender o código entregue.
Versão de entrega	Registrar o commit entregue no prazo. Não incluir alterações posteriores na versão avaliada, salvo autorização do professor.

Quem não souber explicar o código na defesa terá a nota individual do trabalho zerada.

Escolha e aprovação do tema

Sem repetição	Cada tema fica com um único grupo. A reserva depende de aprovação do professor, antes do desenvolvimento.
Proposta para aprovação	Enviar tema, integrantes, link do dataset, pergunta, X, y, tipo de problema e recorte pretendido.
Viabilidade primeiro	Preferir CSV tabular, alvo conhecido e variáveis compreensíveis. Confirmar acesso e licença antes de reservar.

Comunicação em sala ou no Blackboard. Prioridade conforme registro e orientação do professor.

Temas 01 e 02

01 Aprovação de estudantes

Classificação

UCI Student Performance

X: tempo de estudo, reprovações anteriores e apoio escolar.

y: aprovado, definido por G3 ≥ 10.

Excluir G3 de X. Definir o momento da previsão.

02 Evasão universitária

Classificação

UCI Predict Students' Dropout

X: nota de ingresso, idade na matrícula e condição de bolsista.

y: evasão ou conclusão.

Recortar Dropout/Graduate e excluir Enrolled.

Temas 03 e 04

03 Inadimplência e crédito

Classificação

UCI Default of Credit Card Clients

X: limite de crédito, atrasos e pagamentos anteriores.

y: inadimplência no mês seguinte.

Converter a base original para CSV, se necessário.

04 Churn de clientes

Classificação

UCI Iranian Churn

X: falhas de chamadas, reclamações, uso e tempo de contrato.

y: cliente encerrou o serviço (Churn).

Utilizar informações anteriores ao cancelamento.

Temas 05 e 06

05 Preço de imóveis

Regressão

House Prices / Ames Housing

X: área habitável, quartos, ano de construção e bairro.

y: preço de venda (SalePrice).

Começar com poucas variáveis. Avaliar com MAE.

06 Preço de veículos usados

Regressão

Vehicle Dataset / CarDekho

X: ano, quilometragem, combustível e transmissão.

y: preço de venda (selling_price).

Documentar a moeda e tratar categorias.

Temas 07 e 08

07 Atraso de voos

Classificação

BTS Airline On-Time Statistics

X: companhia, mês, dia da semana, horário previsto e distância.

y: chegada com atraso de 15 min ou mais.

Recortar um período. Excluir voos cancelados e dados pós-voo de X.

08 Qualidade de vinhos

Classificação

UCI Wine Quality

X: álcool, acidez volátil, densidade e sulfatos.

y: qualidade alta (quality ≥ 7) ou demais.

Justificar o limite e observar o equilíbrio das classes.

Temas 09 e 10

09 Popularidade de jogos

Classificação

Steam Store Games, de Nik Davis

X: preço, gênero, plataformas e idade mínima.

y: faixa de proprietários (owners) alta ou baixa.

Fixar o corte antes do teste. Excluir avaliações e uso de X.

10 Preço de cursos online

Regressão

Udemy Courses, de Daniel Grijalva

X: duração, número de aulas, nível e área do curso.

y: preço anunciado (price).

Usar apenas cursos pagos e avaliar com MAE.

Temas 11 e 12

11 Doença cardíaca

Classificação

UCI Heart Disease, recorte Cleveland

X: idade, pressão em repouso, colesterol e frequência máxima.

y: presença de doença (num > 0).

Tratar ausentes e apresentar como estudo didático.

12 Diabetes

Classificação

Pima Indians Diabetes / OpenML 37

X: glicose, IMC, idade e pressão arterial.

y: classe de diabetes registrada na base.

Investigar zeros inválidos e limites da população amostrada.

Temas 13 e 14

13 Rotatividade de funcionários

Classificação

IBM HR Analytics Attrition

X: renda mensal, horas extras, tempo na empresa e satisfação.

y: saída do funcionário (Attrition).

Base fictícia para aprendizado. Excluir identificadores.

14 Campanhas de marketing

Classificação

UCI Bank Marketing

X: idade, ocupação, contatos anteriores e empréstimos.

y: adesão ao depósito a prazo (yes/no).

Excluir duração da ligação para prever antes do contato.

Roteiro da demonstração em sala

1. Contexto e dados	Apresentar a pergunta, a fonte, X e y. Mostrar a aplicação iniciando conforme o README.
2. Upload e análise	Enviar um CSV válido, interpretar os três gráficos e demonstrar uma entrada inválida tratada.
3. Modelo e previsão	Comparar os modelos, interpretar métricas e erros e justificar a escolha final. Demonstrar a previsão e discutir limitações.

Checklist para a entrega

Aplicação completa	Upload, Pandas, três gráficos e previsão funcionam. Pelo menos dois modelos foram comparados e a escolha final justificada.
Projeto reproduzível	Outro integrante consegue executar pelo README. CSV de exemplo e dependências estão disponíveis.
Defesa preparada	Todos explicam X/y, limpeza, funcionamento dos modelos, avaliação, erros e integração. Tema e dataset têm aprovação.

Valor: 5,0 pontos. Grupos de 3 a 4 alunos.